Assignment 6: Error Detection using Checksum and CRC

Simiyon Vinscent Samuel L
Register Number: 3122247001062
Submission Date: August 18, 2026
GitHub Repository: https://github.com/blackscythe123/PCN-assign

Aim

To build small C programs that show how a receiver can detect transmission errors without any extra hardware – first using the 1's complement checksum method locally, then using a Cyclic Redundancy Check (CRC) sent for real over a TCP socket. The goal is to actually see a bit error get caught, not just compute a checksum value and call it done.

Part (a): Checksum – End-Around-Carry Addition

`checksum.c` does not need sockets, since part (a) of the brief only asks for sender/receiver logic, not a network transfer – that requirement is satisfied by part (b) instead. The program splits the message into 8-bit words (one word per character), adds them with 1's complement addition, and applies end-around carry: whenever an addition overflows past 8 bits, the overflow bit is wrapped back around and added into the sum instead of being discarded. The final checksum is the 1's complement of that sum.

Listing 1: checksum.c – add_with_end_around_carry(): the core end-around-carry loop, printing every intermediate sum and carry

```
for (int i = 0; i < count; i++) {
    sum = sum + words[i];
    int carry = 0;

    /* end-around carry: if the addition overflowed past 8 bits,
     * wrap the overflow bit back around and add it to the sum */
    if (sum > 0xFF) {
        sum = (sum & 0xFF) + 1;
        carry = 1;
    }

    printf("  step %2d: word = %3u (0x%02X)  ->  sum = %3u (0x%02X)
        carry-out = %d\n",
            i + 1, words[i], words[i], sum & 0xFF, sum & 0xFF, carry);
}
```

The receiver side re-adds the data words plus the checksum word using the exact same function. If nothing was corrupted, that sum comes out as all 1s (255, i.e. 0xFF for an 8-bit word) and the program prints ERROR-FREE. `main()` runs the whole sender+receiver simulation twice on the same message ("NETWORKS"): once untouched, and once with bit 0 of the first transmitted word manually flipped right before the receiver step, so both outcomes show up side by side in the same run.

For the message "NETWORKS", the sender computed a final sum of 127 (0x7F) and a checksum of 128 (0x80). In the clean run the receiver's re-sum came to 255 (0xFF) – all 1s – and was reported ERROR-FREE. In the corrupted run, flipping bit 0 of the first word changed it from 78 to 79, and every subsequent intermediate sum in the receiver's re-addition differed from the clean run; the final sum came to 1 (0x01) instead of 255, and the program correctly reported ERROR DETECTED.

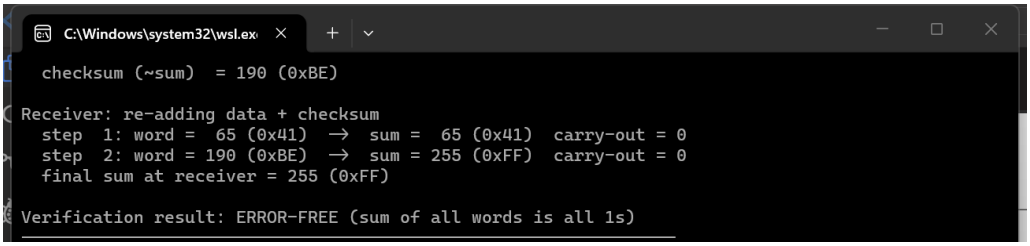

Figure 1: checksum.c clean run: intermediate sums/carries for both the sender and receiver passes, ending in an ERROR-FREE verification.

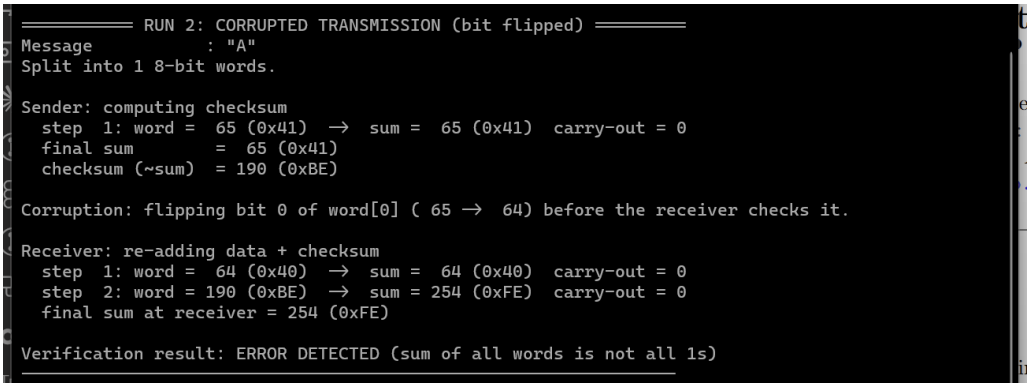

Figure 2: checksum.c corrupted run: the same message with bit 0 of the first word flipped before the receiver re-adds it, ending in an ERROR DETECTED verification.

Part (b): CRC over TCP Sockets

`crc_sender.c` and `crc_receiver.c` are a real TCP client/server pair communicating on port 6063. The sender converts the message to its binary equivalent (8 bits per character), computes the CRC using the generator G(x) = x^4 + x + 1 (bit pattern 10011) via mod-2 polynomial division, and appends the 4 CRC bits to the data before sending the whole frame over the socket. The receiver divides the entire received frame (data + CRC) by the same generator; a zero remainder means the frame is error-free, any non-zero remainder means it was corrupted.

Listing 2: crc_sender.c / crc_receiver.c – mod2_divide(): binary long division by XOR, the core of the CRC computation and verification

```
1  void mod2_divide(const char *dividend, int dividend_len,
2                   const char *generator, int gen_len, char *
                        remainder_out) {
3      char temp[256];
4      strcpy(temp, dividend);
5
6      for (int i = 0; i <= dividend_len - gen_len; i++) {
7          /* only XOR when the leading bit at this position is 1 -
```

```
            * that's how binary long division skips over leading zeros */
            if (temp[i] == '1') {
                for (int j = 0; j < gen_len; j++) {
                    temp[i + j] = (temp[i + j] == generator[j]) ? '0' : '1'
                        ;
                }
            }
        }

        int rem_len = gen_len - 1;
        strncpy(remainder_out, temp + (dividend_len - rem_len), rem_len);
        remainder_out[rem_len] = '\0';
}
```

To demonstrate an error over the real socket connection, `crc_sender` takes an optional `corrupt` argument; when passed, it flips one bit inside the already-framed data (data + CRC) right before sending, and prints exactly which bit changed. The receiver has no idea whether a given run is clean or corrupted – it just divides whatever frame it receives and reports what it finds, which is what makes this a real test of the detection logic rather than a scripted result.

For the message "HELLO" (binary `01001000 01000101 01001100 01001100 01001111`), the sender computed a CRC of `1111`, giving the transmitted frame `01001000 01000101 01001100 01001100 01001111 1111`. In the clean run the receiver divided that frame by the generator and got a remainder of 0000, reporting the frame error-free. In the corrupted run, the sender flipped bit position 3 of the frame (0 to 1) before sending; the receiver divided the altered frame and got a non-zero remainder of 0111, correctly reporting that the frame contained errors.

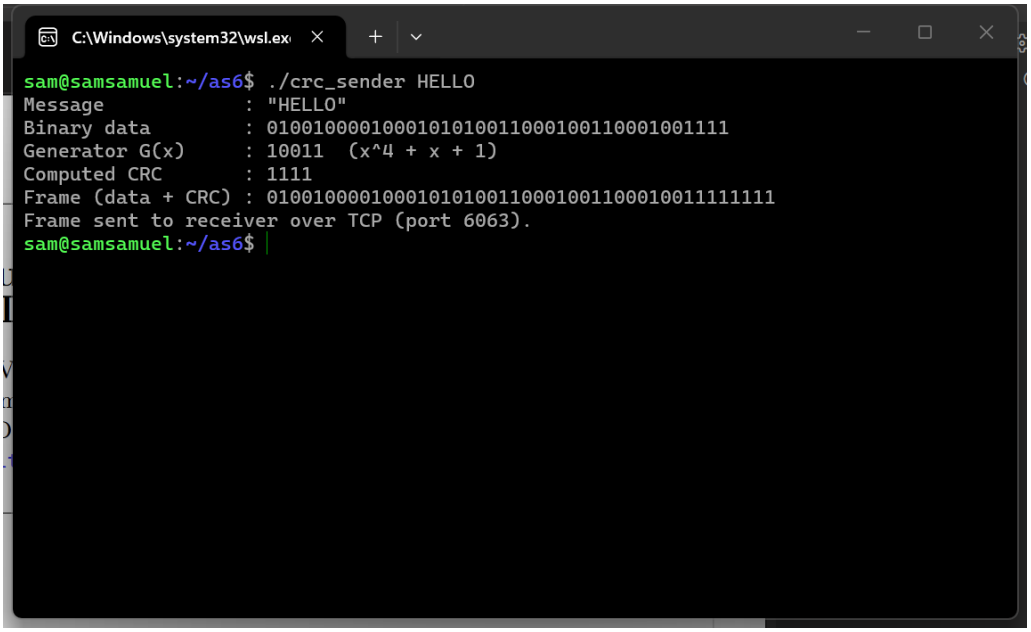

Figure 3: crc_sender.c terminal: binary conversion of "HELLO", the generator 10011, the computed CRC 1111, and the combined frame sent over TCP.

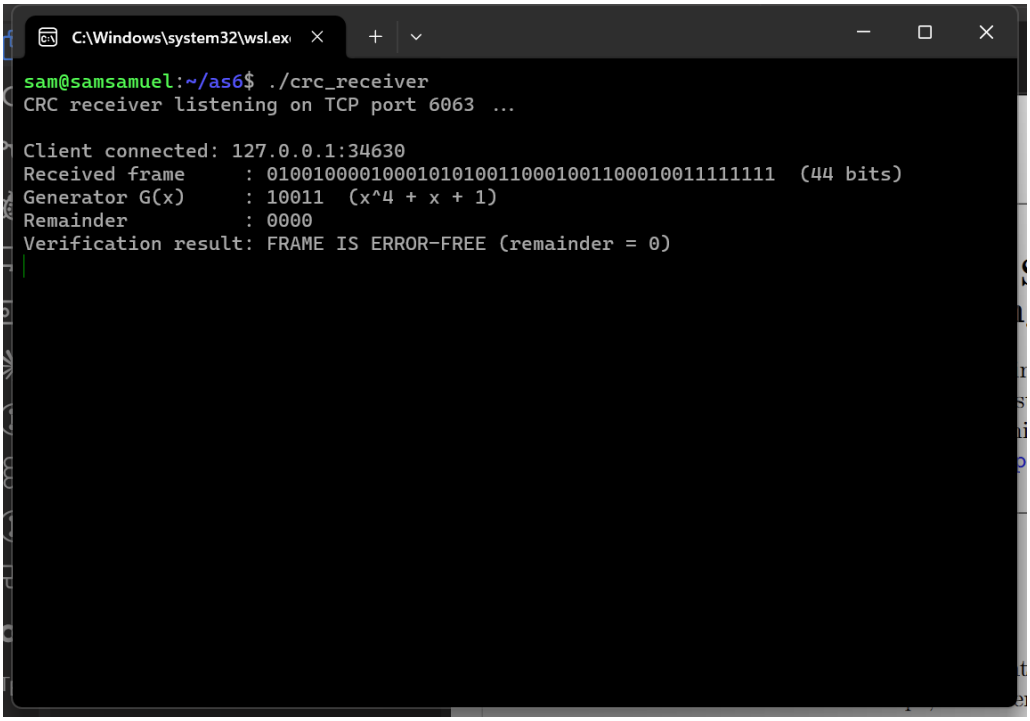

Figure 4: crc_receiver.c terminal for the clean frame: remainder 0000, reported as error-free.

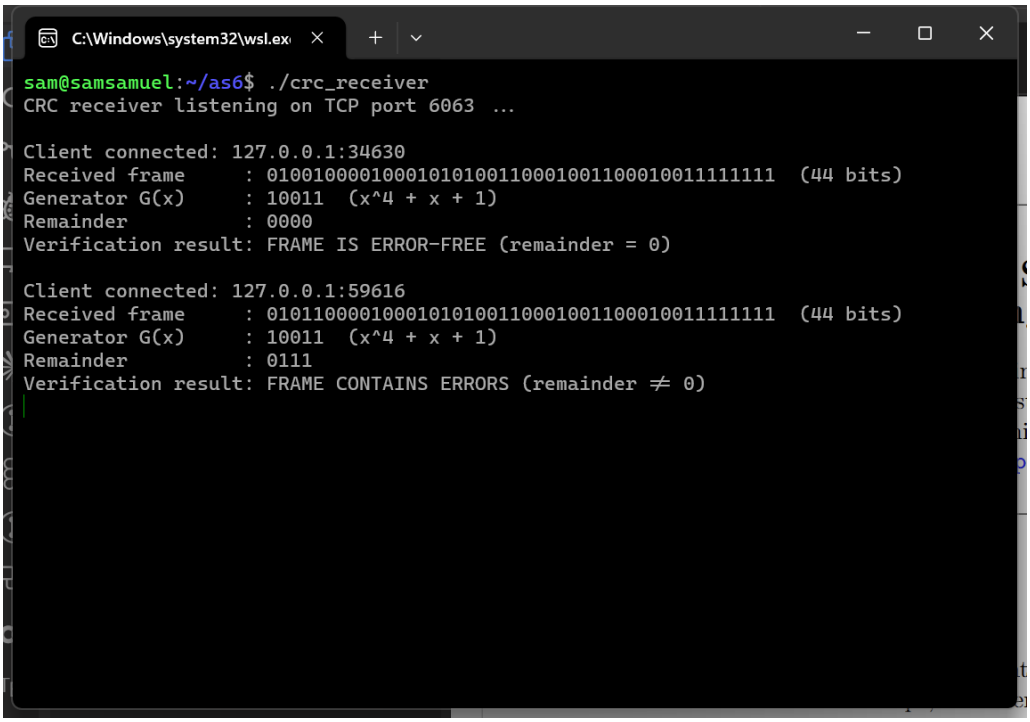

Figure 5: crc_receiver.c terminal for the corrupted frame: non-zero remainder 0111, reported as containing errors.

Learning Outcome

- End-around carry is easy to describe but easy to get wrong in code – the fix is just "if the sum overflows 8 bits, add the overflow bit back in," but forgetting the wrap-back step (or wrapping the whole sum instead of just the excess bit) silently breaks the checksum.
- CRC division looked intimidating on paper but turned out to just be repeated XOR at each '1' bit going left to right – once that clicked, writing `mod2_divide` took a handful of lines, and it is literally the same function on both the sender and receiver.
- Running the CRC pair over a real TCP socket (rather than just calling the same function twice in one program) made it obvious that the receiver genuinely does not know in advance whether a frame is clean or corrupted – it only ever sees a remainder, and everything downstream depends on trusting that remainder is zero.